

Git para Desarrolladores Java

Transición desde SVN con Ejemplos Prácticos

Guía Completa para Desarrolladores con 1 año de experiencia

25 de noviembre de 2025

Índice

1	Introducción: ¿Por qué cambiar de SVN a Git?	4
1.1	Principales diferencias	4
1.2	Comparación SVN vs Git	4
2	Conceptos Fundamentales	4
2.1	Los Tres Estados de Git	4
2.2	Ramas en Git	5
3	Instalación y Configuración Inicial	5
3.1	Configuración básica	5
3.2	Configuración específica para Java	5
4	Comandos Esenciales con Ejemplos	6
4.1	Inicialización y Clonado	6
4.2	Seguimiento de Archivos	6
4.3	Commits	6
5	Flujos de Trabajo Comunes	7
5.1	Git Flow para Equipos Java	7
5.2	Hotfixes	7
6	Archivos .gitignore para Proyectos Java	8
6.1	Contenido típico de .gitignore	8
7	Resolución de Conflictos	9
7.1	Conflictos de Merge	9
8	Comandos Avanzados Útiles	9
8.1	Historial y Navegación	9
8.2	Deshacer Cambios	9
9	Integración con IDEs Java	10
9.1	IntelliJ IDEA	10
9.2	Eclipse	10

10 Mejores Prácticas para Equipos Java	10
10.1 Convenciones de Commits	10
10.2 Estructura de Ramas	11
11 Migración desde SVN	11
11.1 Preparación	11
12 Herramientas Complementarias	11
12.1 GUI Tools	11
12.2 Plataformas de Hosting	12
13 Troubleshooting Común	12
13.1 Problemas Frecuentes	12
14 Ejercicios Prácticos	12
14.1 Ejercicio 1: Flujo Básico	12
14.2 Ejercicio 2: Colaboración	12
15 Referencias y Recursos	13
15.1 Documentación Oficial	13
15.2 Cursos y Tutoriales	13
16 Conclusiones	13
17 Fundamentos Profundos de Git	13
17.1 Arquitectura Interna de Git	13
17.1.1 El Modelo de Datos	14
17.1.2 Hash SHA-1 y Integridad	14
17.2 El Área de Preparación (Staging Area) en Detalle	14
18 Comandos Avanzados y Técnicas Expertas	15
18.1 Manipulación Avanzada del Historial	15
18.1.1 Git Rebase - Reescribir Historia	15
18.1.2 Git Cherry-pick Avanzado	15
18.2 Búsqueda y Depuración Avanzada	16
18.2.1 Git Bisect - Búsqueda Binaria de Bugs	16
18.2.2 Git Grep Avanzado	16
18.3 Configuración Avanzada de Git	17
18.3.1 Configuraciones de Performance	17
18.3.2 Hooks Avanzados	17
19 Técnicas de Colaboración Avanzadas	18
19.1 Estrategias de Merge Avanzadas	18
19.2 Gestión Avanzada de Ramas	18
20 Git para Proyectos Empresariales Java	19
20.1 Monorepos y Submódulos	19
20.2 Git Worktree - Múltiples Working Directories	20
21 Técnicas de Debugging Avanzadas	20

21.1	Git Reflog - Historial de Referencias	20
21.2	Git Stash Avanzado	20
22	Optimización y Performance	21
22.1	Git LFS para Archivos Grandes	21
22.2	Mantenimiento del Repositorio	21
23	Integración con Herramientas Java	22
23.1	Maven y Git	22
23.2	Gradle y Git	22
24	Scripts y Automatización Avanzada	23
24.1	Scripts de Git Personalizados	23
25	Casos de Uso Avanzados Específicos	23
25.1	Migración Compleja desde SVN	23
25.2	Deployment con Git	24

1. Introducción: ¿Por qué cambiar de SVN a Git?

Como desarrollador Java con experiencia en SVN, ya tienes una base sólida en control de versiones. Git te ofrecerá nuevas capacidades que transformarán tu flujo de trabajo:

1.1. Principales diferencias

- **Distribución:** Cada clon es un repositorio completo
- **Ramas:** Extremadamente ligeras y rápidas
- **Offline:** Puedes trabajar sin conexión al servidor
- **Staging Area:** Área intermedia para preparar commits
- **Historia:** Historial completo en tu máquina local

1.2. Comparación SVN vs Git

Aspecto	SVN	Git
Arquitectura	Centralizada	Distribuida
Ramas	Costosas (carpetas)	Ligeras (punteros)
Commits	Directos al servidor	Locales + Push
Trabajo offline	Limitado	Completo
Velocidad	Depende de red	Muy rápido

2. Conceptos Fundamentales

2.1. Los Tres Estados de Git

1. **Working Directory:** Archivos en tu directorio de trabajo
2. **Staging Area:** Archivos preparados para commit
3. **Repository:** Historial de commits confirmados

```
# Working Directory -> Staging Area
git add archivo.java

# Staging Area -> Repository
git commit -m "Mensaje del commit"

# Repository -> Servidor remoto
git push origin main
```

Listing 1: Flujo básico de estados

2.2. Ramas en Git

A diferencia de SVN donde las ramas son carpetas costosas, en Git son simplemente punteros ligeros.

```
# Crear nueva rama
git branch feature/nueva-funcionalidad

# Cambiar a la rama
git checkout feature/nueva-funcionalidad

# Crear y cambiar en un comando
git checkout -b feature/otra-funcionalidad

# Listar ramas
git branch

# Fusionar rama
git checkout main
git merge feature/nueva-funcionalidad
```

Listing 2: Manejo básico de ramas

3. Instalación y Configuración Inicial

3.1. Configuración básica

```
# Configurar identidad
git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@empresa.com"

# Configurar editor
git config --global core.editor "code --wait"

# Ver configuraci n
git config --list
```

Listing 3: Configuración inicial de Git

3.2. Configuración específica para Java

```
# Ignorar archivos compilados por defecto
git config --global core.excludesfile ~/.gitignore_global

# Configurar diff para archivos Java
git config --global diff.tool vscode

# Configurar merge tool
git config --global merge.tool vscode
```

Listing 4: Configuraciones útiles para desarrollo Java

4. Comandos Esenciales con Ejemplos

4.1. Inicialización y Clonado

```
# Inicializar repositorio nuevo
git init mi-proyecto-java

# Clonar repositorio existente
git clone https://github.com/usuario/proyecto-java.git

# Clonar con nombre específico
git clone https://github.com/usuario/proyecto.git mi-proyecto
```

Listing 5: Crear y clonar repositorios

4.2. Seguimiento de Archivos

```
# Ver estado de archivos
git status

# Agregar archivos específicos
git add src/main/java/com/empresa/MiClase.java

# Agregar todos los archivos Java
git add "*.java"

# Agregar todo
git add .

# Ver diferencias
git diff
git diff --staged
```

Listing 6: Gestión de archivos

4.3. Commits

```
# Commit básico
git commit -m "Implementar validación de usuario"

# Commit con descripción detallada
git commit -m "Implementar validación de usuario

- Agregar clase ValidadorUsuario
- Implementar validación de email
- Agregar tests unitarios
- Actualizar documentación"

# Commit que incluye modificaciones
git commit -a -m "Corregir bug en login"
```

Listing 7: Creación de commits

5. Flujos de Trabajo Comunes

5.1. Git Flow para Equipos Java

```
# 1. Actualizar rama principal
git checkout main
git pull origin main

# 2. Crear rama para nueva funcionalidad
git checkout -b feature/JIRA-123-validacion-usuarios

# 3. Desarrollar y hacer commits
# ... modificar archivos ...
git add .
git commit -m "JIRA-123: Implementar validaci n b sica"

# 4. Subir rama al servidor
git push -u origin feature/JIRA-123-validacion-usuarios

# 5. Crear Pull Request en la plataforma
# 6. Despu s de aprobaci n , fusionar
git checkout main
git pull origin main
git branch -d feature/JIRA-123-validacion-usuarios
```

Listing 8: Flujo típico en proyecto Java

5.2. Hotfixes

```
# Crear rama de hotfix desde main
git checkout main
git pull origin main
git checkout -b hotfix/correccion-critica-login

# Hacer la correcc i n
# ... modificar archivos ...
git add .
git commit -m "Hotfix: Corregir NPE en login"

# Subir y fusionar urgentemente
git push -u origin hotfix/correccion-critica-login
# Crear PR urgente para revisi n r pida
```

Listing 9: Manejo de correcciones urgentes

6. Archivos .gitignore para Proyectos Java

6.1. Contenido típico de .gitignore

```
# Archivos compilados
*.class
*.jar
*.war
*.ear

# IDEs
.idea/
.vscode/
*.iml
*.iws
*.ipr

# Eclipse
.metadata
.classpath
.project
.settings/

# Maven
target/
pom.xml.tag
pom.xml.releaseBackup
pom.xml.versionsBackup

# Gradle
.gradle/
build/

# Logs
*.log
logs/

# OS
.DS_Store
Thumbs.db

# Spring Boot
application-local.properties
application-dev.properties
```

Listing 10: .gitignore para proyectos Java

7. Resolución de Conflictos

7.1. Conflictos de Merge

```
# 1. Intentar merge
git merge feature/nueva-funcionalidad

# Si hay conflictos, Git mostrar :
# Auto-merging src/main/java/com/empresa/Usuario.java
# CONFLICT (content): Merge conflict in Usuario.java

# 2. Ver archivos con conflictos
git status

# 3. Editar archivos manualmente
# Buscar marcadores <<<<<< ===== >>>>>>

# 4. Marcar como resueltos
git add src/main/java/com/empresa/Usuario.java

# 5. Completar merge
git commit -m "Resolver conflicto en clase Usuario"
```

Listing 11: Resolver conflictos paso a paso

8. Comandos Avanzados Útiles

8.1. Historial y Navegación

```
# Ver historial
git log --oneline --graph --all

# Ver cambios en archivo específico
git log -p src/main/java/com/empresa/Usuario.java

# Buscar en historial
git log --grep="JIRA-123"

# Ver qui n modific cada l nea
git blame src/main/java/com/empresa/Usuario.java
```

Listing 12: Explorar historial

8.2. Deshacer Cambios

```
# Descartar cambios no confirmados
git checkout -- archivo.java

# Deshacer staging
```

```
git reset HEAD archivo.java

# Deshacer ltimo commit (mantener cambios)
git reset --soft HEAD~1

# Deshacer ltimo commit (descartar cambios)
git reset --hard HEAD~1

# Revertir commit p blico
git revert abc123def
```

Listing 13: Deshacer diferentes tipos de cambios

9. Integración con IDEs Java

9.1. IntelliJ IDEA

- **VCS Menu:** Acceso completo a operaciones Git
- **Git Tool Window:** Vista del historial y ramas
- **Commit Tool Window:** Staging y commits visuales
- **Merge Conflicts:** Herramienta visual de resolución

9.2. Eclipse

- **EGit Plugin:** Integración completa con Git
- **Git Repositories View:** Gestión de repositorios
- **Git Staging View:** Control del staging area
- **History View:** Exploración del historial

10. Mejores Prácticas para Equipos Java

10.1. Convenciones de Commits

```
# Formato: tipo(scope): descripci n

feat(auth): implementar validaci n JWT
fix(login): corregir NPE en validaci n
docs(api): actualizar documentaci n REST
test(user): agregar tests de integraci n
refactor(db): optimizar consultas de usuario
```

Listing 14: Formato de mensajes de commit

10.2. Estructura de Ramas

- **main/master**: Código de producción
- **develop**: Integración de funcionalidades
- **feature/***: Nuevas funcionalidades
- **hotfix/***: Correcciones urgentes
- **release/***: Preparación de versiones

11. Migración desde SVN

11.1. Preparación

```
# 1. Crear archivo de mapeo de usuarios
echo "usuario_svn = Nombre Completo <email@empresa.com>" > users.
txt

# 2. Clonar repositorio SVN
git svn clone --authors-file=users.txt \
http://svn.empresa.com/repo/proyecto \
--trunk=trunk --branches=branches --tags=tags

# 3. Convertir ramas y tags
cd proyecto
git for-each-ref refs/remotes/origin | cut -d/ -f3- | \
grep -v '^tags/' | while read branchname; do
    git branch "$branchname" "refs/remotes/origin/$branchname"
    git branch -r -d "origin/$branchname"
done

# 4. Crear repositorio Git remoto
git remote add origin https://github.com/empresa/proyecto.git
git push -u origin main
```

Listing 15: Pasos para migrar desde SVN

12. Herramientas Complementarias

12.1. GUI Tools

- **SourceTree**: GUI gratuita multiplataforma
- **GitKraken**: GUI moderna con funciones avanzadas
- **GitHub Desktop**: Simple para repositorios GitHub
- **TortoiseGit**: Integración con Windows Explorer

12.2. Plataformas de Hosting

- **GitHub:** Más popular, gran ecosistema
- **GitLab:** CI/CD integrado, versión self-hosted
- **Bitbucket:** Integración con Atlassian (Jira)
- **Azure DevOps:** Integración con Microsoft Stack

13. Troubleshooting Común

13.1. Problemas Frecuentes

```
# Problema: "fatal: refusing to merge unrelated histories"
git pull origin main --allow-unrelated-histories

# Problema: Archivos grandes accidentalmente agregados
git filter-branch --force --index-filter \
  'git rm --cached --ignore-unmatch archivo-grande.jar' \
  --prune-empty --tag-name-filter cat -- --all

# Problema: Cambiar mensaje del ltimo commit
git commit --amend -m "Nuevo mensaje"

# Problema: Olvid agregar archivo al ltimo commit
git add archivo-olvidado.java
git commit --amend --no-edit
```

Listing 16: Soluciones a problemas comunes

14. Ejercicios Prácticos

14.1. Ejercicio 1: Flujo Básico

1. Crear repositorio para proyecto Java
2. Configurar .gitignore apropiado
3. Crear estructura Maven/Gradle básica
4. Hacer commits incrementales
5. Crear y fusionar rama de funcionalidad

14.2. Ejercicio 2: Colaboración

1. Simular conflicto de merge
2. Practicar resolución manual
3. Usar herramientas visuales del IDE

4. Crear Pull Request
5. Revisar código de compañero

15. Referencias y Recursos

15.1. Documentación Oficial

- Pro Git Book: <https://git-scm.com/book>
- Git Reference: <https://git-scm.com/docs>
- GitHub Guides: <https://guides.github.com/>

15.2. Cursos y Tutoriales

- Atlassian Git Tutorial: <https://www.atlassian.com/git/tutorials>
- Learn Git Branching: <https://learngitbranching.js.org/>
- GitHub Learning Lab: <https://lab.github.com/>

16. Conclusiones

La transición de SVN a Git puede parecer compleja inicialmente, pero las ventajas son significativas:

- **Velocidad:** Operaciones locales instantáneas
- **Flexibilidad:** Ramas ligeras y múltiples flujos de trabajo
- **Distribución:** Cada desarrollador tiene historial completo
- **Integración:** Mejor soporte en IDEs modernos
- **Ecosistema:** Herramientas y servicios abundantes

Con práctica constante y los ejemplos de esta guía, dominarás Git rápidamente y mejorarás significativamente tu productividad como desarrollador Java.

17. Fundamentos Profundos de Git

17.1. Arquitectura Interna de Git

Git no es solo un sistema de control de versiones, es un sistema de archivos dirigido por contenido con una interfaz VCS.

17.1.1. El Modelo de Datos

```
# Git almacena 4 tipos de objetos:

# 1. BLOB (Binary Large Object) - Contenido de archivos
# SHA-1: da39a3ee5e6b4b0d3255bfef95601890afd80709 (archivo vacío)

# 2. TREE - Estructura de directorios
# Contiene referencias a blobs y otros trees

# 3. COMMIT - Instantánea del proyecto
# Referencia a tree + metadatos (autor, fecha, mensaje)

# 4. TAG - Referencia a commit con metadata adicional
```

Listing 17: Estructura interna de objetos Git

17.1.2. Hash SHA-1 y Integridad

```
# Cada objeto Git tiene un hash SHA-1 único
# Basado en el contenido, garantiza integridad

# Ejemplo de hash de commit
git log --oneline
# a1b2c3d Mensaje del commit

# Ver el objeto completo
git cat-file -p a1b2c3d
# tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
# author Juan Perez <juan@empresa.com> 1700000000 +0200
# committer Juan Perez <juan@empresa.com> 1700000000 +0200
#
# Mensaje del commit
```

Listing 18: Verificación de integridad con SHA-1

17.2. El Área de Preparación (Staging Area) en Detalle

```
# Ver contenido del índice
git ls-files --stage

# Añadir partes específicas de un archivo
git add -p archivo.java

# Añadir líneas específicas interactivamente
git add -i

# Resetear archivo del staging area
git reset HEAD archivo.java
```

```
# Ver diferencias entre working dir, staging y repository
git diff                    # working vs staging
git diff --staged           # staging vs repository
git diff HEAD               # working vs repository
```

Listing 19: Manipulación avanzada del staging area

18. Comandos Avanzados y Técnicas Expertas

18.1. Manipulación Avanzada del Historial

18.1.1. Git Rebase - Reescribir Historia

```
# Rebase interactivo para limpiar commits
git rebase -i HEAD~3

# Opciones disponibles en rebase interactivo:
# pick    = usar commit tal como est
# reword  = usar commit pero editar mensaje
# edit    = usar commit pero parar para enmendar
# squash  = usar commit pero fusionar con el anterior
# fixup   = como squash pero descartar mensaje
# drop    = remover commit

# Rebase con estrategia espec fica
git rebase -X theirs feature-branch

# Rebase preservando merges
git rebase --rebase-merges main

# Continuar rebase despu s de resolver conflictos
git rebase --continue

# Abortar rebase
git rebase --abort
```

Listing 20: Rebase interactivo y avanzado

18.1.2. Git Cherry-pick Avanzado

```
# Cherry-pick m ltiples commits
git cherry-pick commit1 commit2 commit3

# Cherry-pick rango de commits
git cherry-pick start-commit..end-commit

# Cherry-pick sin hacer commit autom tico
git cherry-pick -n commit-hash
```

```
# Cherry-pick editando el mensaje
git cherry-pick -e commit-hash

# Cherry-pick firmando el commit
git cherry-pick -S commit-hash
```

Listing 21: Cherry-pick con opciones avanzadas

18.2. Búsqueda y Depuración Avanzada

18.2.1. Git Bisect - Búsqueda Binaria de Bugs

```
# Iniciar bisect
git bisect start

# Marcar commit actual como malo
git bisect bad

# Marcar commit conocido bueno
git bisect good v1.0.0

# Git automáticamente selecciona commit del medio
# Probar y marcar como bueno o malo
git bisect good # si funciona
git bisect bad  # si tiene el bug

# Automatizar con script
git bisect run test-script.sh

# Finalizar bisect
git bisect reset
```

Listing 22: Encontrar bugs con bisect

18.2.2. Git Grep Avanzado

```
# Buscar en todo el repositorio
git grep -n "patr n"

# Buscar en commits específicos
git grep "patr n" commit-hash

# Buscar con regex
git grep -E "patr n|otro-patr n"

# Buscar considerando contexto
git grep -A 3 -B 3 "patr n"

# Buscar en archivos específicos
git grep "patr n" -- "*.java"
```



```
# Buscar ignorando case
git grep -i "patr n"

# Contar ocurrencias
git grep -c "patr n"
```

Listing 23: Búsqueda avanzada en historial

18.3. Configuración Avanzada de Git

18.3.1. Configuraciones de Performance

```
# Configurar compresión
git config --global pack.compression 9

# Configurar ventana de delta
git config --global pack.window 250

# Habilitar preload index
git config --global core.preloadindex true

# Configurar parallel index
git config --global core.untrackedCache true

# Configurar fsmonitor (para repositorios grandes)
git config --global core.fsmonitor true

# Configurar split-index
git config --global core.splitIndex true
```

Listing 24: Optimizaciones de rendimiento

18.3.2. Hooks Avanzados

```
# Pre-commit hook para validar código Java
#!/bin/bash
# .git/hooks/pre-commit

# Ejecutar checkstyle
mvn checkstyle:check
if [ $? -ne 0 ]; then
    echo "Checkstyle falló. Corrige los errores antes del commit"
    exit 1
fi

# Ejecutar tests unitarios
mvn test
if [ $? -ne 0 ]; then
    echo "Tests fallan. Corrige antes del commit."
    exit 1
```

```
fi

# Pre-push hook para validar branch
#!/bin/bash
# .git/hooks/pre-push

protected_branch='main'
current_branch=$(git symbolic-ref HEAD | sed -e 's,.*\/\(.*\),\1,')

if [ $current_branch = $protected_branch ]; then
    echo "Push directo a $protected_branch no permitido."
    exit 1
fi
```

Listing 25: Scripts de automatización con hooks

19. Técnicas de Colaboración Avanzadas

19.1. Estrategias de Merge Avanzadas

```
# Merge con estrategia específica
git merge -X ours feature-branch      # En conflicto, preferir
    nuestra versión
git merge -X theirs feature-branch    # En conflicto, preferir su
    versión

# Merge ignorando whitespace
git merge -X ignore-space-change feature-branch

# Merge con estrategia personalizada
git merge -s recursive -X patience feature-branch

# Merge octopus (múltiples branches)
git merge branch1 branch2 branch3

# Crear merge commit sin fast-forward
git merge --no-ff feature-branch

# Merge sin commit automático
git merge --no-commit feature-branch
```

Listing 26: Estrategias de merge personalizadas

19.2. Gestión Avanzada de Ramas

```
# Ver ramas merged/no-merged
git branch --merged main
git branch --no-merged main
```

```
# Eliminar ramas remotas hu rfanas
git remote prune origin

# Renombrar rama actual
git branch -m nuevo-nombre

# Copiar rama
git branch nueva-rama rama-existente

# Crear rama desde commit espec fico
git branch nueva-rama commit-hash

# Seguir rama remota
git branch --set-upstream-to=origin/feature-branch

# Ver informaci n detallada de ramas
git branch -vv
```

Listing 27: Operaciones complejas con ramas

20. Git para Proyectos Empresariales Java

20.1. Monorepos y Submódulos

```
# A adir subm dulo
git submodule add https://github.com/empresa/libreria.git libs/
  libreria

# Clonar con subm dulos
git clone --recurse-submodules proyecto.git

# Actualizar subm dulos
git submodule update --remote

# Inicializar subm dulos despu s de clon
git submodule init
git submodule update

# Trabajar dentro de subm dulo
cd libs/libreria
git checkout master
# hacer cambios
git commit -am "Actualizar librer a"

# Actualizar referencia en proyecto principal
cd ../..
git add libs/libreria
git commit -m "Actualizar subm dulo librer a"
```

Listing 28: Gestión de proyectos grandes

20.2. Git Worktree - Múltiples Working Directories

```
# Crear worktree adicional
git worktree add ../hotfix-workspace hotfix/critical-bug

# Listar worktrees
git worktree list

# Trabajar en paralelo
cd ../hotfix-workspace
# hacer cambios de hotfix

# En workspace principal seguir con feature
# hacer cambios de feature

# Eliminar worktree
git worktree remove ../hotfix-workspace
```

Listing 29: Trabajar en múltiples ramas simultáneamente

21. Técnicas de Debugging Avanzadas

21.1. Git Reflog - Historial de Referencias

```
# Ver historial de HEAD
git reflog

# Ver reflog de rama específica
git reflog show feature-branch

# Recuperar commit "perdido"
git checkout commit-hash-from-reflog
git branch recovered-branch

# Limpiar reflog (cuidado!)
git reflog expire --expire=30.days refs/heads/main
git gc --prune=30.days.ago
```

Listing 30: Recuperar commits perdidos

21.2. Git Stash Avanzado

```
# Stash con mensaje
git stash push -m "WIP: implementando feature X"

# Stash incluyendo archivos no rastreados
git stash -u

# Stash archivos específicos
```

```
git stash push path/to/file.java

# Ver contenido de stash
git stash show -p stash@{0}

# Aplicar stash sin eliminarlo
git stash apply stash@{0}

# Crear rama desde stash
git stash branch nueva-rama stash@{0}

# Limpiar stash
git stash clear
```

Listing 31: Gestión avanzada de stash

22. Optimización y Performance

22.1. Git LFS para Archivos Grandes

```
# Instalar Git LFS
git lfs install

# Trackear archivos por extensi n
git lfs track "*.jar"
git lfs track "*.war"

# Trackear archivos por path
git lfs track "docs/assets/*"

# Ver archivos trackeados
git lfs track

# Migrar archivos existentes
git lfs migrate import --include="*.jar"

# Ver informaci n de LFS
git lfs ls-files
git lfs status
```

Listing 32: Gestión de archivos grandes

22.2. Mantenimiento del Repositorio

```
# Garbage collection manual
git gc --aggressive --prune=now

# Verificar integridad
git fsck --full
```

```
# Ver estadísticas del repositorio
git count-objects -v

# Limpiar referencias remotas
git remote prune origin

# Repack para optimizar
git repack -a -d --depth=250 --window=250

# Limpiar archivos no rastreados
git clean -fdx
```

Listing 33: Optimización y limpieza

23. Integración con Herramientas Java

23.1. Maven y Git

```
# Plugin de release de Maven con Git
mvn release:prepare release:perform

# Configuración en pom.xml
<scm>
  <connection>scm:git:git://github.com/empresa/proyecto.git</connection>
  <developerConnection>scm:git:ssh://git@github.com/empresa/proyecto.git</developerConnection>
  <url>https://github.com/empresa/proyecto</url>
</scm>

# Git ignore específico para Maven
echo "target/" >> .gitignore
echo "/*.releaseBackup" >> .gitignore
echo "release.properties" >> .gitignore
```

Listing 34: Integración con Maven

23.2. Gradle y Git

```
# Plugin de release para Gradle
plugins {
  id 'net.researchgate.release' version '2.8.1'
}

# Configurar versión desde Git
version = getVersionFromGit()

def getVersionFromGit() {
```

```
def stdout = new ByteArrayOutputStream()
exec {
    commandLine 'git', 'describe', '--tags', '--always'
    standardOutput = stdout
}
return stdout.toString().trim()
}
```

Listing 35: Integración con Gradle

24. Scripts y Automatización Avanzada

24.1. Scripts de Git Personalizados

```
# Configurar alias avanzados
git config --global alias.cleanup-branches '!git branch --merged
main | grep -v "main\|develop" | xargs -n 1 git branch -d'

git config --global alias.find-merge '!sh -c '\''merge=$(git rev-
list $1..HEAD --ancestry-path | tail -1) && git show $merge'\''
-'

git config --global alias.contributors 'shortlog -sn --all --no-
merges'

git config --global alias.changelog 'log --oneline --decorate --
graph --since="1 week ago"'

# Script para crear feature branch
#!/bin/bash
# git-new-feature
FEATURE_NAME=$1
if [ -z "$FEATURE_NAME" ]; then
    echo "Uso: git new-feature nombre-feature"
    exit 1
fi

git checkout develop
git pull origin develop
git checkout -b feature/$FEATURE_NAME
git push -u origin feature/$FEATURE_NAME
```

Listing 36: Aliases y scripts avanzados

25. Casos de Uso Avanzados Específicos

25.1. Migración Compleja desde SVN

```
# Crear mapeo completo de usuarios
svn log -q | awk -F '|' '{sub(/ /, "", $2); sub(" $", "", $2); print $2 = "$2" <"$2"@empresa.com>"}' | sort -u > authors-transform.txt

# Migración con todas las ramas y tags
git svn clone --stdlayout --authors-file=authors-transform.txt \
  --prefix=svn/ http://svn.empresa.com/repo/proyecto

# Convertir tags SVN a tags Git reales
for t in $(git for-each-ref --format='%(refname:short)' refs/remotes/svn/tags); do
  git tag ${t/svn\//tags\//} $t
  git branch -r -d $t
done

# Limpiar referencias SVN
git for-each-ref --format='delete %(refname)' refs/remotes/svn |
  git update-ref --stdin
git remote rm svn 2>/dev/null
```

Listing 37: Migración avanzada con preservación de historia

25.2. Deployment con Git

```
# Git deployment con hooks
#!/bin/bash
# post-receive hook en servidor

# Checkout automático al recibir push
cd /var/www/aplicacion
git --git-dir=/var/repo/proyecto.git --work-tree=/var/www/
  aplicacion checkout -f

# Ejecutar build
mvn clean package -DskipTests

# Reiniciar servicio
systemctl restart aplicacion.service

echo "Deployment completado exitosamente"
```

Listing 38: Estrategias de deployment

Con esta extensión profunda, ahora tienes una comprensión completa de Git desde sus fundamentos internos hasta las técnicas más avanzadas para proyectos empresariales Java.